



PROCESO DE APRENDIZAJE 1

NRC: 4682

CURSO: Algoritmo y Estructura de Datos Basados en IA

DOCENTE: Wilder Julio Espinoza Bravo

INTEGRANTES

- Mariagracia Cadillo Jiménez 100%
- Diana Carolina Romero Pariana 100%
- Cristian Martin Correa Barriga 100%
- Mauricio Del Carpio Torres 100%

Lima, Perú

2026

1. INTRODUCCIÓN

En esta presente evaluación se desarrollará una propuesta para organizar información en relación con talleres estudiantiles utilizando estructuras lineales estáticas. Se trabajan conceptos de algoritmos, datos, matrices y arreglos unidimensionales, además de operaciones de ordenamiento, búsqueda, inserción y recorrido de datos.

2. ACTIVIDAD 1 — ANÁLISIS DEL PROBLEMA Y SELECCIÓN DE ESTRUCTURA

2.1 Diferencia entre estructura estática y dinámica

Una estática se establece previamente y normalmente no presenta cambios durante la ejecución del programa, por otro lado en la dinámica se le puede modificar su tamaño durante en la ejecución del programa ya sea agregar o eliminar elementos según las necesidades. Un ejemplo son las listas enlazadas.

2.2 ¿Por qué utilizar arreglos y matrices?

Porque resulta adecuado utilizar los arreglos y matrices en el caso que trabaja con cantidades de estudiantes previamente organizadas.

- Los arreglos permiten almacenar información en posiciones identificadas mediante índices.

- Las matrices representan la información organizada mediante columnas y filas.

2.3 Relación entre dato, algoritmo y estructura de datos

- Dato: es la información que se almacena en el sistema.
- Estructura de datos: Es la forma que organizamos y almacenamos los datos.
- Algoritmo: es la agrupación de pasos que permite procesar los datos almacenados.

3. ACTIVIDAD 2 — MODELADO Y OPERACIONES CON VECTORES (diana)

3.1 Vector inicial

Se utilizará un arreglo unidimensional `int [ ]`, para almacenar la cantidad de estudiantes inscritos en cada taller.

```
int[] inscritos = {28, 15, 34, 21, 19, 40, 12, 26};
```

codigo:

```
public static void vectorInicial(int[] vector) {  
  
    System.out.println("\n3.1 VECTOR INICIAL");  
  
    System.out.println("-----");  
  
    System.out.println("Cantidad de inscritos por taller:");  
  
    for (int i = 0; i < vector.length; i++) {  
        System.out.println(  
            "Taller " + (i + 1) + ": " + vector[i] + "  
inscritos"  
        );  
    }  
  
    System.out.println("\nVector:");  
    System.out.println(Arrays.toString(vector));  
}
```

3.1 VECTOR INICIAL

```
-----  
Cantidad de inscritos por taller:  
Taller 1: 28 inscritos  
Taller 2: 15 inscritos  
Taller 3: 34 inscritos  
Taller 4: 21 inscritos  
Taller 5: 19 inscritos  
Taller 6: 40 inscritos  
Taller 7: 12 inscritos  
Taller 8: 26 inscritos
```

3.2 Representación gráfica

El programa genera una pequeña grafica utilizando (*). Por ejemplo, el taller con 12 inscritos tendrá 12 asteriscos.

```
public static void representacionGrafica(int[] vector) {  
  
    System.out.println("\n3.2 REPRESENTACIÓN GRÁFICA");  
  
    System.out.println("-----");  
  
    for (int i = 0; i < vector.length; i++) {  
  
        System.out.print("Taller " + (i + 1) + " | ");  
  
        for (int j = 0; j < vector[i]; j++) {  
            System.out.print("*");  
        }  
  
        System.out.println(" (" + vector[i] + ")");  
    }  
}
```

```
3.2 REPRESENTACIÓN GRÁFICA  
-----  
Taller 1 | ***** (28)  
Taller 2 | ***** (15)  
Taller 3 | ***** (34)  
Taller 4 | ***** (21)  
Taller 5 | ***** (19)  
Taller 6 | ***** (40)  
Taller 7 | ***** (12)  
Taller 8 | ***** (26)
```

3.3 Algoritmo para encontrar el mayor y menor

El programa encontrará:

- Mayor: 40 inscritos -> Taller 6
- Menor: 12 inscritos -> Taller 7

```
public static void encontrarMayorYMenor(int[] vector) {

    System.out.println("\n3.3 MAYOR Y MENOR");

    System.out.println("-----");

    int mayor = vector[0];
    int menor = vector[0];

    int posicionMayor = 0;
    int posicionMenor = 0;

    for (int i = 1; i < vector.length; i++) {

        if (vector[i] > mayor) {
            mayor = vector[i];
            posicionMayor = i;
        }

        if (vector[i] < menor) {
            menor = vector[i];
            posicionMenor = i;
        }
    }

    System.out.println(
        "Mayor cantidad de inscritos: " + mayor +
        " estudiantes"
    );

    System.out.println(
        "Se encuentra en el Taller " +
        (posicionMayor + 1)
    );

    System.out.println(
        "Menor cantidad de inscritos: " + menor +
        " estudiantes"
    );

    System.out.println(
        "Se encuentra en el Taller " +
        (posicionMenor + 1)
    );
}
```

```
);  
}
```

3.3 MAYOR Y MENOR

```
-----  
Mayor cantidad de inscritos: 40 estudiantes  
Se encuentra en el Taller 6  
Menor cantidad de inscritos: 12 estudiantes  
Se encuentra en el Taller 7
```

3.4 Algoritmo para insertar un valor

Para demostrar la integración, se agregará el valor 30 en la posición 4.

El vector pasa de:

[28, 15, 34, 21, 19, 40, 12, 26]

a:

[28, 15, 34, 21, 30, 19, 40, 12, 26]

```
public static int[] insertarValor(int[] vector, int valor, int  
posicion) {  
  
    System.out.println("\n3.4 INSERTAR UN VALOR");  
  
    System.out.println("-----");  
  
    System.out.println("Vector original:");  
    System.out.println(Arrays.toString(vector));  
  
    System.out.println(  
        "Se insertará el valor " + valor +  
        " en la posición " + posicion  
    );  
  
    // Crear un nuevo vector con una posición adicional  
    int[] nuevoVector = new int[vector.length + 1];  
  
    // Copiar los elementos antes de la posición  
    for (int i = 0; i < posicion; i++) {
```

```

        nuevoVector[i] = vector[i];
    }

    // Insertar el nuevo valor
    nuevoVector[posicion] = valor;

    // Desplazar los elementos restantes
    for (int i = posicion; i < vector.length; i++) {
        nuevoVector[i + 1] = vector[i];
    }

    System.out.println("Vector después de insertar:");
    System.out.println(Arrays.toString(nuevoVector));

    return nuevoVector;
}

```

3.4 INSERTAR UN VALOR

```

-----
Vector original:
[28, 15, 34, 21, 19, 40, 12, 26]
Se insertar el valor 30 en la posición 4
Vector después de insertar:
[28, 15, 34, 21, 30, 19, 40, 12, 26]

```

3.5 Algoritmo de ordenamiento

En este caso utilizaré el método Burbuja para ordenar de menor a mayor.

```

public static void ordenarVector(int[] vector) {

    System.out.println("\n3.5 ALGORITMO DE ORDENAMIENTO");

    System.out.println("-----");

    System.out.println("Vector antes de ordenar:");
    System.out.println(Arrays.toString(vector));

    // Ordenamiento Burbuja
    for (int i = 0; i < vector.length - 1; i++) {

        for (int j = 0; j < vector.length - 1 - i; j++) {

```

```

        if (vector[j] > vector[j + 1]) {

            int auxiliar = vector[j];
            vector[j] = vector[j + 1];
            vector[j + 1] = auxiliar;

        }
    }

    System.out.println("Vector ordenado de menor a mayor:");
    System.out.println(Arrays.toString(vector));
}

```

3.5 ALGORITMO DE ORDENAMIENTO

```

-----
Vector antes de ordenar:
[28, 15, 34, 21, 30, 19, 40, 12, 26]
Vector ordenado de menor a mayor:
[12, 15, 19, 21, 26, 28, 30, 34, 40]

```

3.6 Costo aproximado del ordenamiento

En nuestro caso usaremos el método burbuja, que compara elementos y los intercambia cuando están en el orden incorrecto. Por ejemplo:

Tenemos el vector: [28, 15, 34, 21, 19, 40, 12, 26]

Después de insertar 30: [28, 15, 34, 21, 30, 19, 40, 12, 26]

Tenemos 9 elementos.

MEJOR CASO:

El mejor caso ocurre cuando el vector ya está ordenado. Si utilizamos el método, podemos detectar que no hubo ningún intercambio y detener el algoritmo.

Costo aproximado: $O(n)$

PEOR CASO:

El peor caso ocurre cuando los elementos están ordenados de mayor a menor. En este caso prácticamente todos los elementos necesitan ser intercambiados y el algoritmo realiza muchas comparaciones.

Costo aproximado: $O(n^2)$ o sea $n(n - 1) / 2$

```
public static void costoOrdenamiento(int[] vector) {

    System.out.println("\n3.6 COSTO APROXIMADO DEL ORDENAMIENTO");

    System.out.println("-----");

    int n = vector.length;

    // MEJOR CASO
    System.out.println("Mejor caso:");
    System.out.println("El vector ya esta ordenado.");
    System.out.println("Costo aproximado: O(n)");

    // PEOR CASO
    System.out.println("\nPeor caso:");

    int comparaciones = 0;

    for (int i = 0; i < n - 1; i++) {

        for (int j = 0; j < n - 1 - i; j++) {

            comparaciones++;

        }

    }

    System.out.println("Cantidad de elementos: " + n);
    System.out.println("Comparaciones aproximadas: " + comparaciones);
    System.out.println("Costo aproximado: O(n²)");

    System.out.println(
        "\nEn el mejor caso, el vector ya esta ordenado " +
        "y se necesita menos trabajo."
    );

    System.out.println(
```

```

        "En el peor caso, el vector esta desordenado " +
        "y se realizan muchas comparaciones e intercambios."
    );
}
}

```

3.6 COSTO APROXIMADO DEL ORDENAMIENTO

Mejor caso:

El vector ya esta ordenado.

Costo aproximado: $O(n)$

Peor caso:

Cantidad de elementos: 9

Comparaciones aproximadas: 36

Costo aproximado: $O(n^2)$

En el mejor caso, el vector ya esta ordenado y se necesita menos trabajo.

En el peor caso, el vector esta desordenado y se realizan muchas comparaciones e intercambios.

4. ACTIVIDAD 3 — MATRICES Y RECORRIDO COMPLETO DE DATOS

4.1 Propuesta de matriz

```

{20, 15, 0, 25, 10},
{ 0, 18, 22, 0, 30},
{12, 0, 19, 14, 0},
{28, 24, 0, 16, 20}

```

4.2 Índices

Filas (i): Representan las Aulas del edificio (índices 0 a 3).

Columnas (j): Representan los Bloques Horarios (índices 0 a 4).

Uso: El acceso `ocupacion[i][j]` indica la cantidad exacta de estudiantes asignados al Aula $i+1$ en el Horario $j+1$.

4.3 Total de estudiantes por aula

Para calcular el total de estudiantes por aula, se realiza un recorrido sumando los valores de cada fila:

- Aula 1: $20 + 15 + 0 + 25 + 10 = 70$ estudiantes
- Aula 2: $0 + 18 + 22 + 0 + 30 = 70$ estudiantes
- Aula 3: $12 + 0 + 19 + 14 + 0 = 45$ estudiantes
- Aula 4: $28 + 24 + 0 + 16 + 20 = 88$ estudiantes

4.4 Total de estudiantes por horario

Para obtener la afluencia en cada bloque horario, se realiza una suma por columnas:

- Horario 1: $20 + 0 + 12 + 28 = 60$ estudiantes
- Horario 2: $15 + 18 + 0 + 24 = 57$ estudiantes
- Horario 3: $0 + 22 + 19 + 0 = 41$ estudiantes
- Horario 4: $25 + 0 + 14 + 16 = 55$ estudiantes
- Horario 5: $10 + 30 + 0 + 20 = 60$ estudiantes

4.5 Celda con mayor ocupación

Al evaluar todos los elementos de la matriz, el valor máximo encontrado es:

- Mayor ocupación: 30 estudiantes
- Ubicación: Aula 2, Horario 5 (Posición en la matriz: `ocupacion[1][4]`).

4.6 ¿Por qué se recorren varias posiciones?

Se deben recorrer múltiples posiciones porque las matrices son estructuras bidimensionales. Para realizar cualquier cálculo de consolidación (como sumar filas/columnas o buscar el elemento con mayor/menor valor), el algoritmo debe evaluar individualmente la intersección de cada celda `[i][j]` mediante ciclos anidados y así garantizar la revisión completa del conjunto de datos.

5. ACTIVIDAD 4 — MATRICES ESPECIALES Y DECISIÓN TÉCNICA

5.1 Matriz cuadrada

Una matriz cuadrada es aquella que tiene la misma cantidad de filas y columnas.

5.2 Matriz poco densa

Una matriz poco densa es aquella en la que la mayoría de sus posiciones contiene valores 0 o no contienen información de mayor relevancia, mientras que solo algunas posiciones contienen datos.

Ejemplo:

```
[20 0 0 0]
[ 0 0 15 0]
[ 0 0 0 0]
[ 0 18 0 0]
```

5.3 Ejemplo aplicado al caso académico

Matriz completa de 4 aulas x 5 bloques (20 celdas)

	B1	B2	B3	B4	B5
Aula 1	[20	0	0	0	0]
Aula 2	[0	0	15	0	0]
Aula 3	[0	0	0	0	0]
Aula 4	[0	18	0	0	0]

Solo 3 de las 20 celdas tienen datos (15%), y el 85% son ceros.
Representación compacta, con índices desde 0:

Fila	Columna	Valor
0	0	20
1	2	15
3	1	18

Pseudocódigo para construirlo:

Para i desde 0 hasta 3:

Para j desde 0 hasta 4:

Si $\text{matriz}[i][j] \neq 0$:

Agregar (i, j, $\text{matriz}[i][j]$) a la lista

5.4 Justificación

- ★ Se evita almacenar y procesar posiciones sin información.
- ★ La matriz completa guarda 20 valores. La compacta guarda 3 registros de 3 datos (9 valores), así que se usa menos de la mitad de la memoria.
- ★ La ventaja crece si el sistema se amplía a muchos horarios y aulas, debido a que la cantidad de ceros crece más rápido que los datos reales.
- ★ Si casi todas las aulas estuvieran ocupadas en todos los horarios, la matriz completa sería mejor, porque cada dato compacto necesita 3 valores y el ahorro desaparecería.